

实验五报告

唐宇恩 251220100

2026 年 5 月 20 日

1 只读存储器实验

1.1 整体方案设计

端口定义：

端口名称	端口功能
ASCII-CODE	输入的 ASCII 码
CodeErr	输入的 ASCII 码是否错误
row1 row16	输出的 16 行点阵数据

我使用一个 `addr_cal` 子电路来计算输入的 ASCII 码对应的 ROM 首地址和 `CodeErr` 信号，输入 `ASCII-CODE`，输出 `Addr` 和 `CodeErr`。然后使用连续 16 个 `read` 子电路计算每个点阵数据的地址，并从 ROM 中读取数据输出到 `row1 row16`，输入为 `Addr_in` 和 `in`（代表当前地址是否需要加 1），输出为 `Addr_out` 和 `out`（代表当前地址的数据）。

1.2 实验电路图

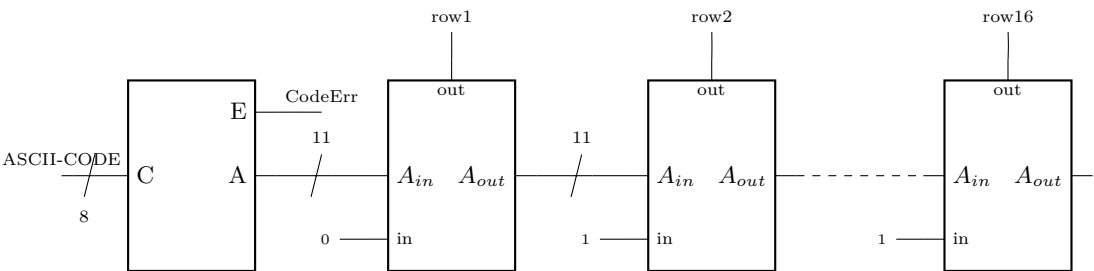


图 1: 原理图

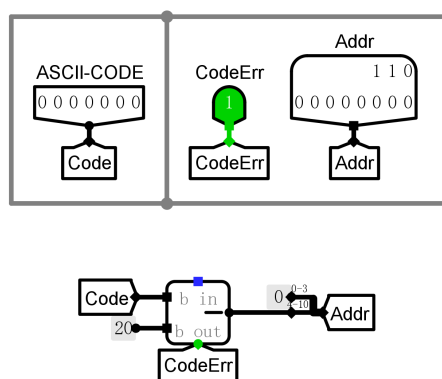


图 2: addr_cal 子电路电路图

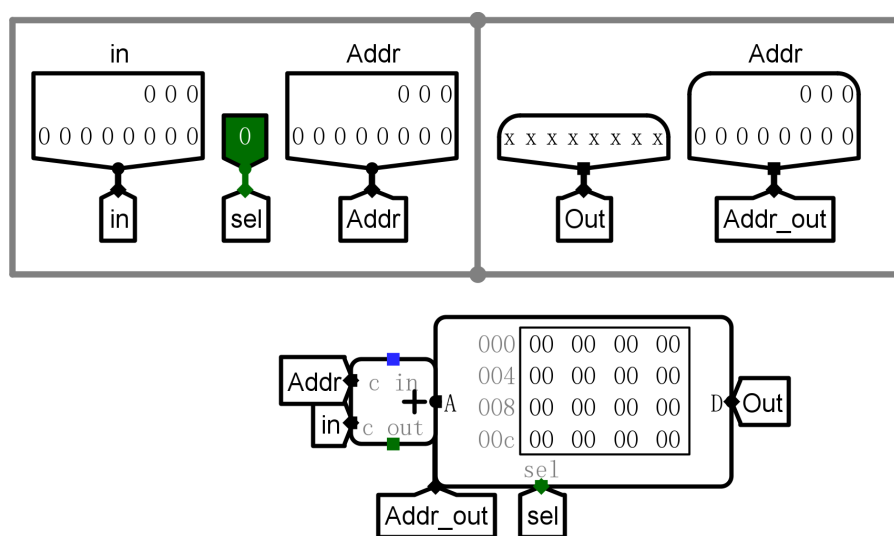


图 3: read 子电路电路图

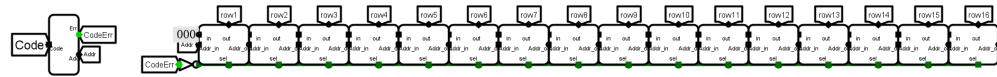
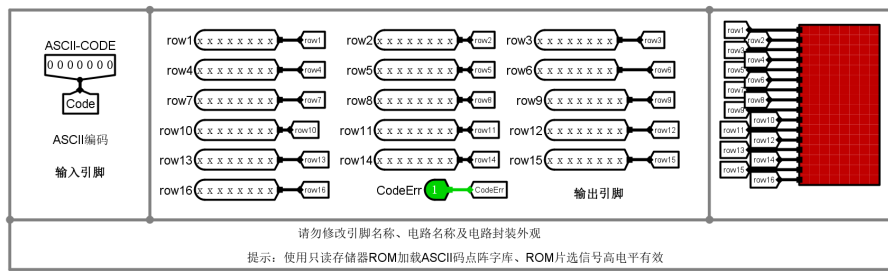


图 4: 主电路图

1.3 仿真测试

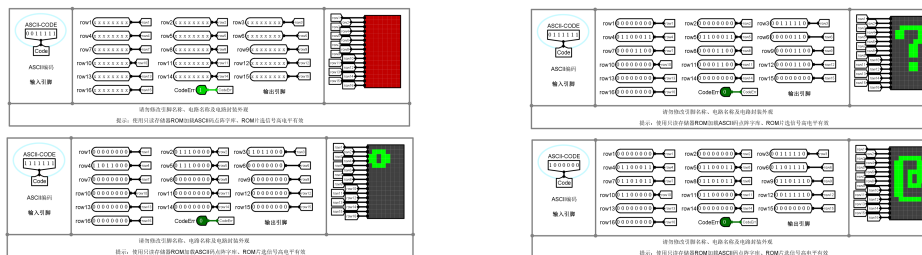


图 5: 仿真测试

1.4 错误分析

本次实验中，没有遇到错误。

2 数据存储器实验

2.1 整体方案设计

端口定义：

端口名称	端口功能
Addr	输入的地址
MemOp	内存控制信号
Din	输入的数据
WE	写使能信号
DataOut	输出的数据
CLK	时钟信号
CLR	异步清零信号
AlignErr	地址对齐错误信号

我用一个 decode 子电路来生成片选信号和 AlignErr 信号，输入 Addr 和 MemOp，输出 SEL[3:0] 和 AlignErr。用一个 io 子电路来处理输入和输出，输入 Addr，MemOp，Din 和 out[3:0] 信号，输出真正的 in 和 Dout 信号连接 RAM。

2.2 实验电路图

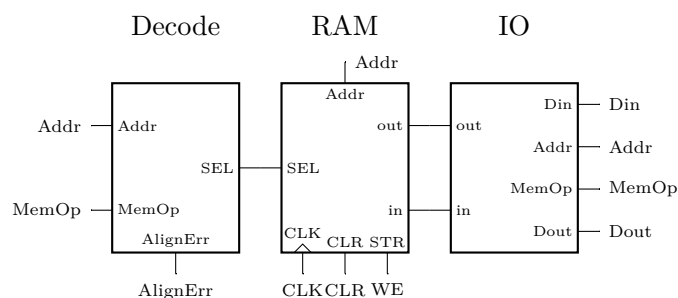


图 6: 数据存储器原理图

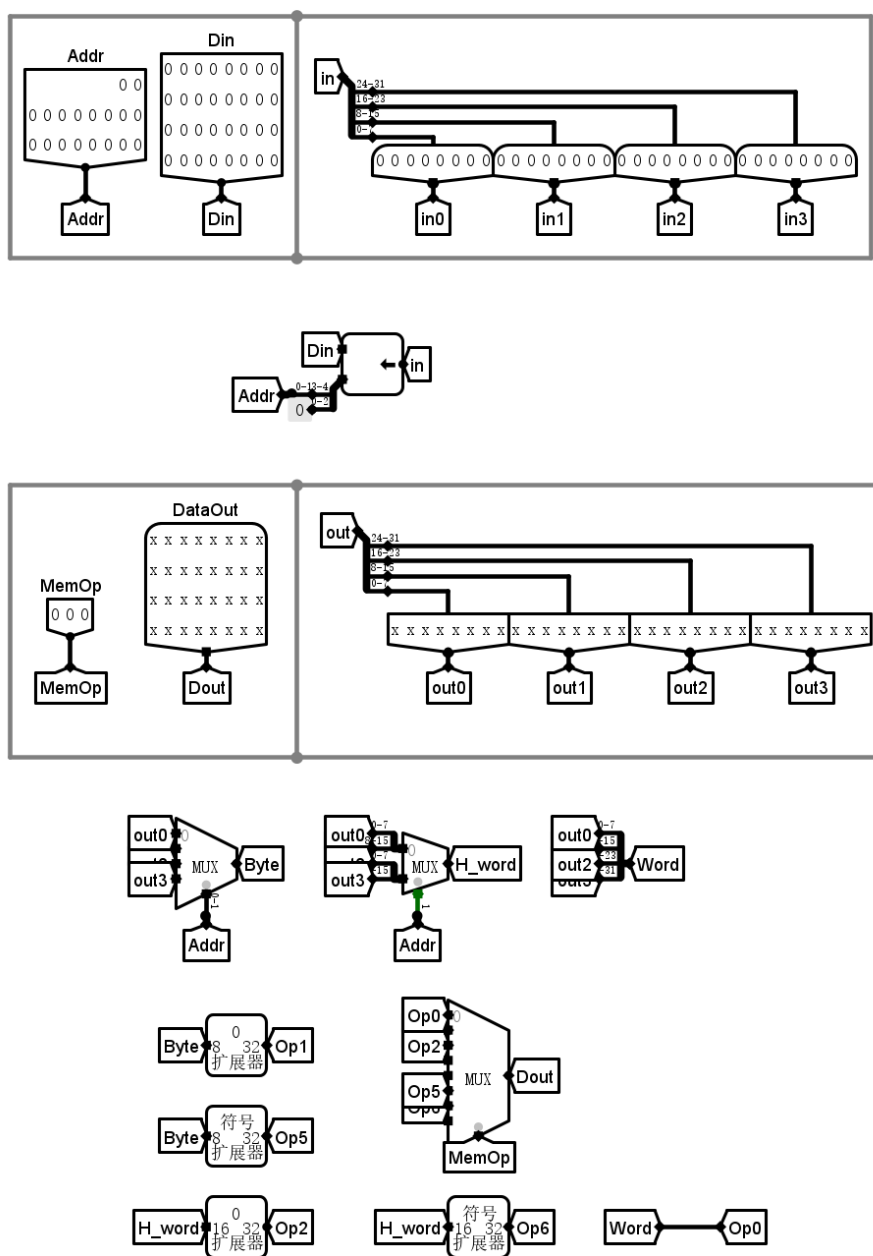


图 7: io 子电路电路图

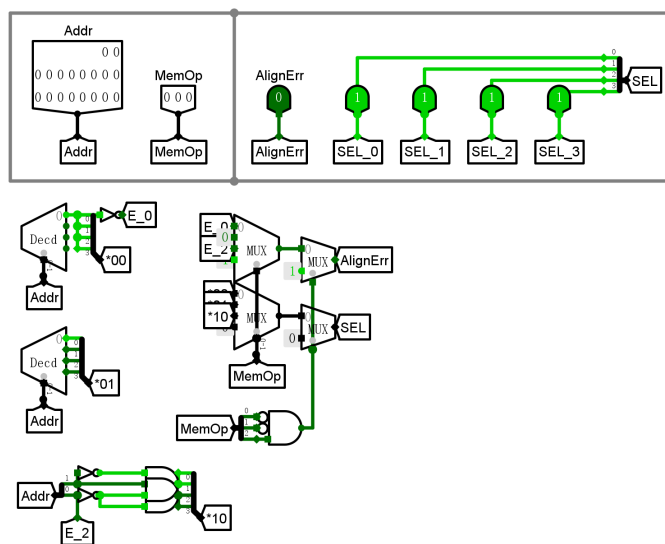


图 8: decode 子电路电路图

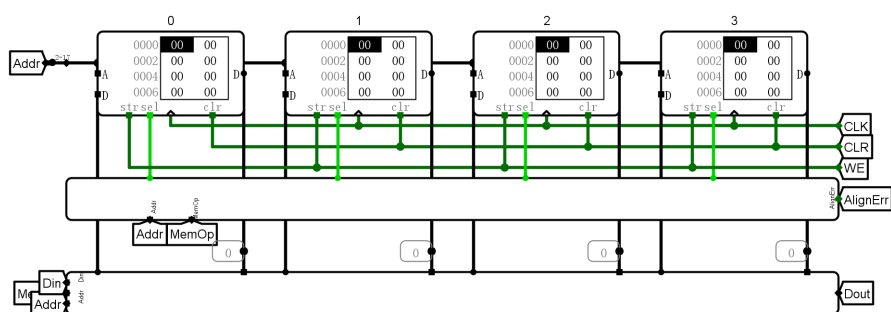
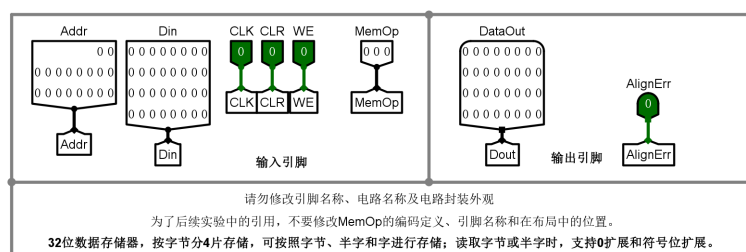


图 9: 主电路图

2.3 仿真测试

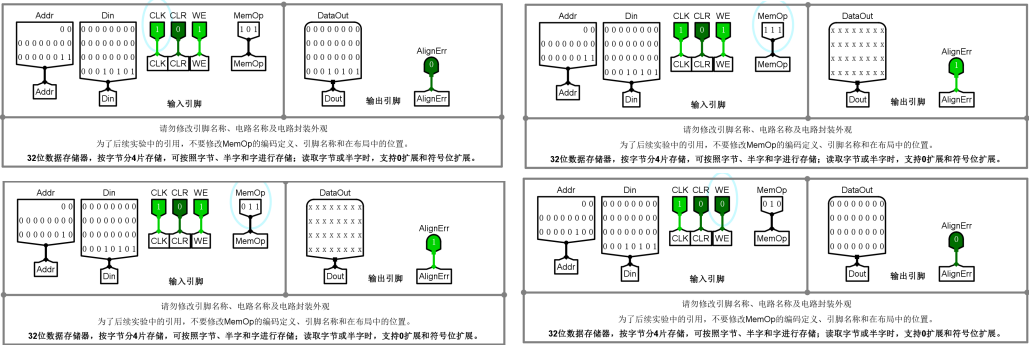


图 10: 仿真测试

2.4 错误分析

一开始 Dout 想当然地使用移位器再连接扩展器处理，但是会遇到高位 x 输出被扩展为 0 的问题。改为现在的实现。

3 取指令部件实验

3.1 整体方案设计

端口定义：

端口名称	端口功能
initAddr	初始化地址
imm	立即数
BusA	输入寄存器的值
CLK	时钟信号
Reset	异步复位信号
NPCASrc,NPCBSrc	选择加法器 A,B 端输入信号
PC	程序计数器值
IR	指令地址值

3.2 实验电路图

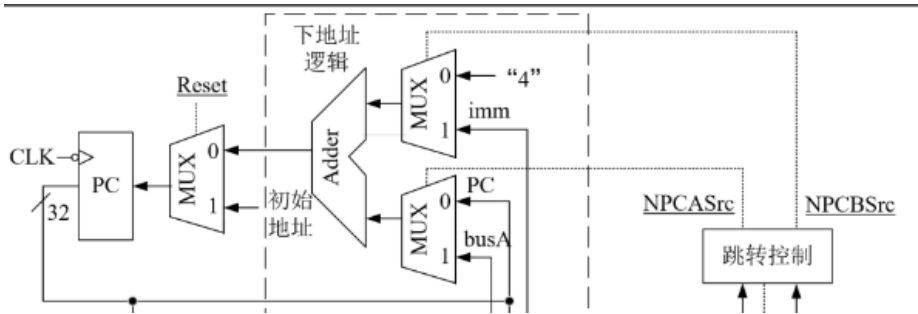


图 11: 取指令部件原理图

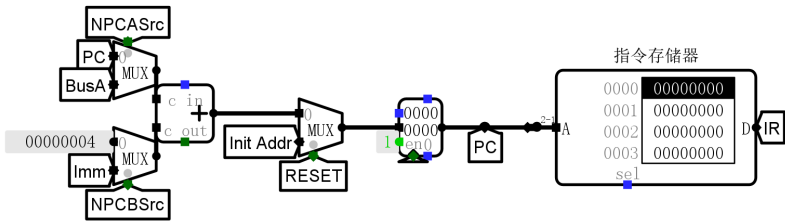
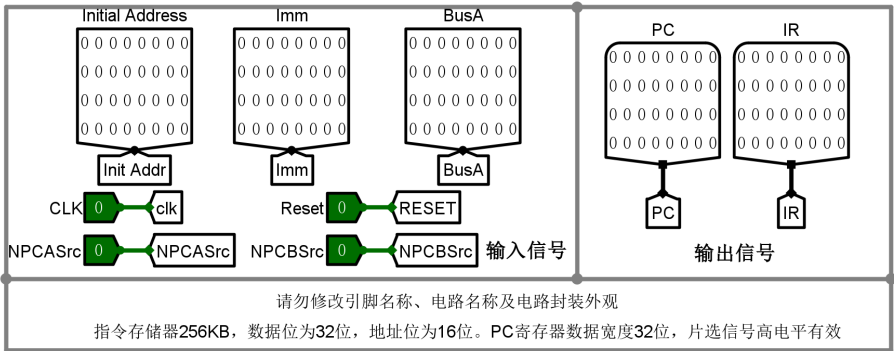


图 12: 取指令部件主电路图

3.3 仿真测试

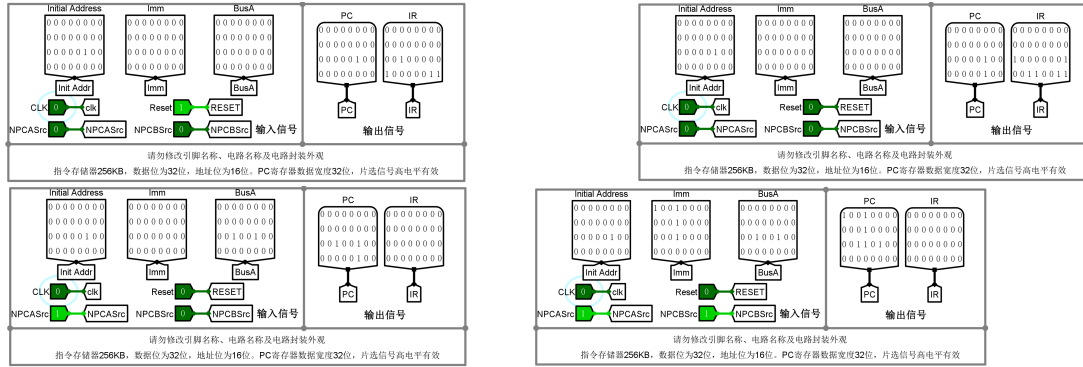


表 1: IFU 测试数据表

输入							输出（边沿信号有效前）	
序号	Reset	Initial Address	Imm	BusA	NPCASrc	NPCBSrc	PC	IR
1	1	0x400	0	0	0	0	0x400	0x2083
2	0	0	0	0	0	0	0x400	0x2083
3	0	0	0	0	0	0	0x404	0x8133
4	0	0	0x7f8	0	0	1	0x408	0x106213
5	0	0	0x20c	0	0	1	0xc00	0x413
6	1	0x1500	0	0	0	0	0xe0c	0xdeadbeef
7	0	0	0	0	0	0	0x1500	0x413
8	0	0	0	0	0	0	0x1504	0x9117
9	0	0	0	0x3fbc0	1	1	0x1508	0xffc10113
10	0	0	0	0	0	0	0x3fbc0	0x413
11	0	0	0x1fc	0	0	1	0x3fbc4	0x9117
12	0	0	0	0x150c	1	1	0x3fbc0	0x40513
13	0	0	0x0c00	0	1	1	0x150c	0x39c000ef
14	0	0	0	0	0	0	0xc00	0x413
15	0	0	0	0	0	0	0xc04	0x9117
16	0	0	0xfffffc8	0	0	1	0xc08	0xffc10113

3.4 错误分析

本次实验中，没有遇到错误。

4 取操作数部件实验

4.1 整体方案设计

端口定义：

端口名称	端口功能
Instr	输入指令
Extop	符号扩展操作类型
PC	程序计数器值
IR	输入指令地址值
CLK	时钟信号
BusW	写回寄存器的值
ALUASrc	ALU A 端输入选择信号
ALUBSrc	ALU B 端输入选择信号
RegWr	寄存器写使能信号
DataA	ALU A 端输入值
DataB	ALU B 端输入值
BusA,BusB	输出寄存器的值

4.2 实验电路图

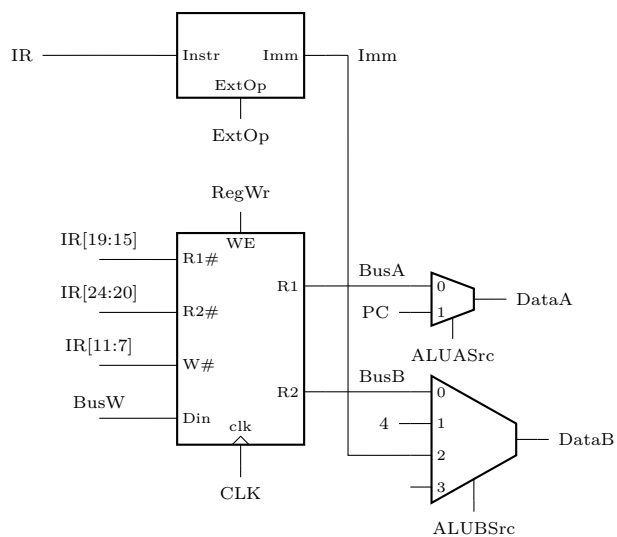


图 14: 取操作数部件原理图

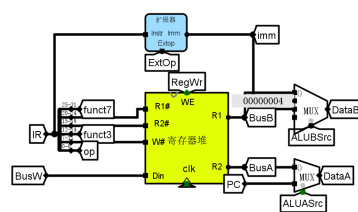
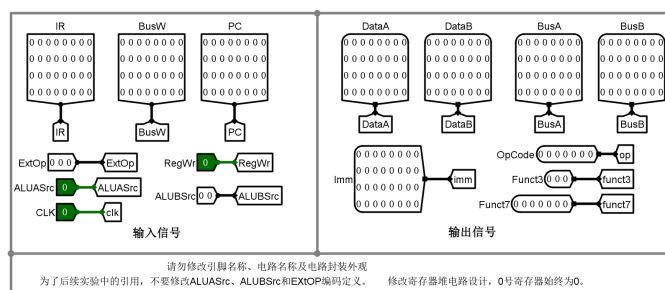


图 15: 主电路图

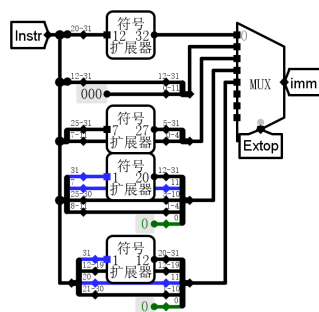
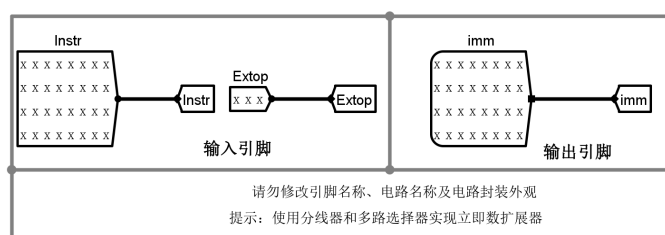


图 16: 立即数扩展电路图

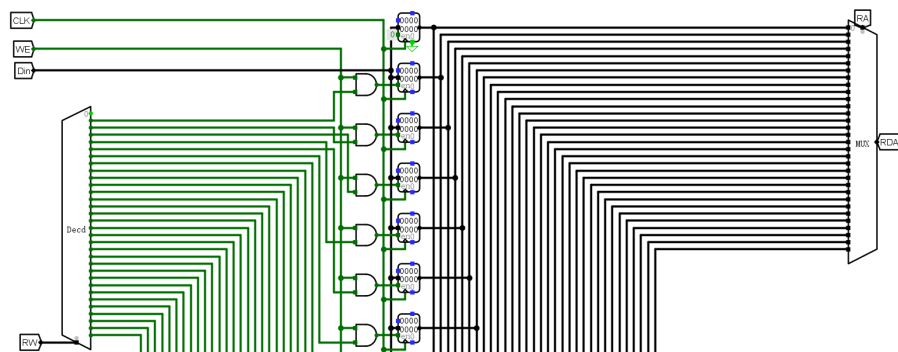
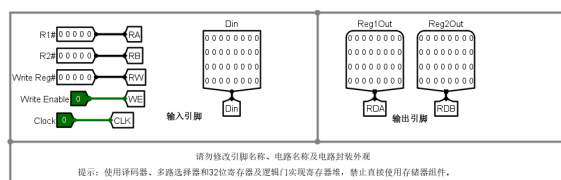


图 17: 寄存器堆电路图

4.3 仿真测试

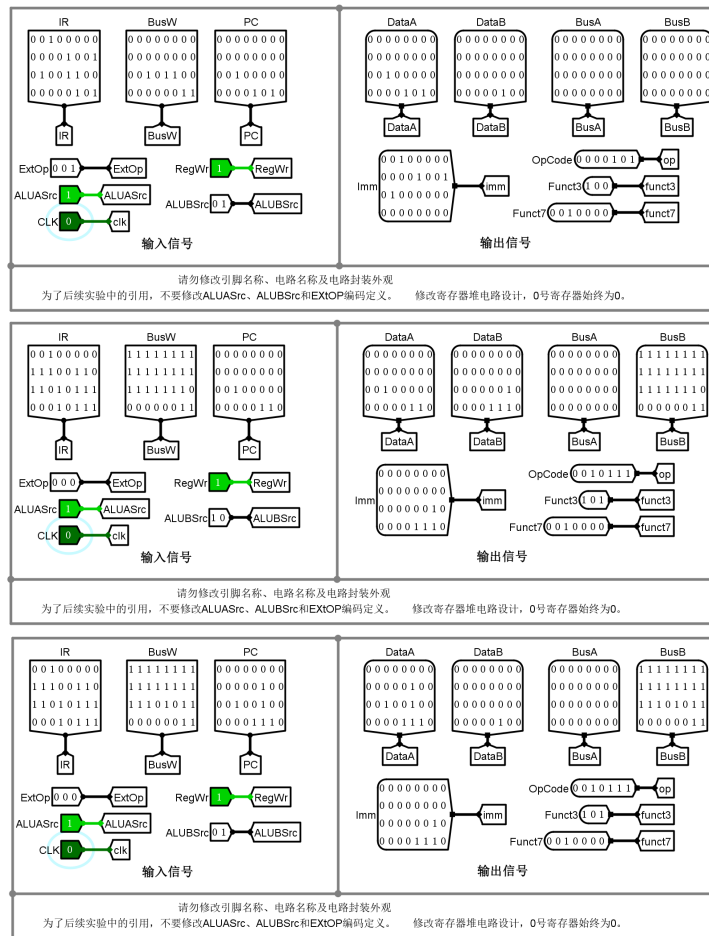


表 2: 取操作数部件输入输出数据列表

PC	IR	BusW	控制信号				DataA	DataB
0	fedca2b7	fedca000	ExtOp=1	RegWr=1	ALUASrc=0	ALUBSrc=2	0x0	0xfedca000
4	f9c28293	fedc9f9c	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=2	0xfedca000	0xfffff9c
8	01006013	10	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=2	0x0	0x10
c	06502223	64	ExtOp=2	RegWr=0	ALUASrc=0	ALUBSrc=2	0x0	0x64
10	06400303	ffff9c	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=2	0x0	0x64
14	06601423	68	ExtOp=2	RegWr=0	ALUASrc=0	ALUBSrc=2	0x0	0x68
18	06405383	9f9c	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=2	0x0	0x64
1c	06701623	6c	ExtOp=2	RegWr=0	ALUASrc=0	ALUBSrc=2	0x0	0x6c
20	4042d413	ffedc9f9	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=2	0xfedc9f9c	0x4
24	006444b3	123665	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=0	0xffedc9f9	0xfffff9c
28	00649533	50000000	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=0	0x123665	0xfffff9c
2c	008505b3	4fedc9f9	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=0	0x50000000	0xffedc9f9
30	00b2a633	1	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=0	0xfedc9f9c	0x4fedc9f9
34	00b2b6b3	0	ExtOp=0	RegWr=1	ALUASrc=0	ALUBSrc=0	0xfedc9f9c	0x4fedc9f9
38	40b287b3	aeed5a3	ExtOp=2	RegWr=0	ALUASrc=0	ALUBSrc=0	0xfedc9f9c	0x4fedc9f9
3c	06f02823	70	ExtOp=3	RegWr=0	ALUASrc=0	ALUBSrc=2	0x0	0x70
40	0067c263	1	ExtOp=3	RegWr=0	ALUASrc=0	ALUBSrc=0	0xaeed5a3	0xfffff9c
44	0067d263	1	ExtOp=4	RegWr=1	ALUASrc=1	ALUBSrc=0	0xaeed5a3	0xfffff9c
48	0040086f	4c	ExtOp=0	RegWr=1	ALUASrc=1	ALUBSrc=1	0x48	0x4
4c	004808e7	50	ExtOp=1	RegWr=1	ALUASrc=1	ALUBSrc=1	0x4c	0x4
50	00001917	1050	ExtOp=0	RegWr=0	ALUASrc=0	ALUBSrc=3	0x50	0x1000

4.4 错误分析

本次实验没有遇到错误。

5 数据通路实验

5.1 整体方案设计

端口定义：

端口名称	端口功能
Initial Address	初始地址
ExtOp	符号扩展操作类型
RegWr	寄存器写使能
MemWr	存储器写使能
MemtoReg	存储器到寄存器选择
ReSet	复位信号
MemOp	存储器操作类型
ALUASrc	ALU 操作数 A 选择
ALUBSrc	ALU 操作数 B 选择
ALUctr	ALU 控制信号
BandJ	分支跳转信号
PC	程序计数器
BusW	数据总线
Din	输入数据
NPCASrc	下一条指令地址选择 A
NPCBSrc	下一条指令地址选择 B

5.2 实验电路图

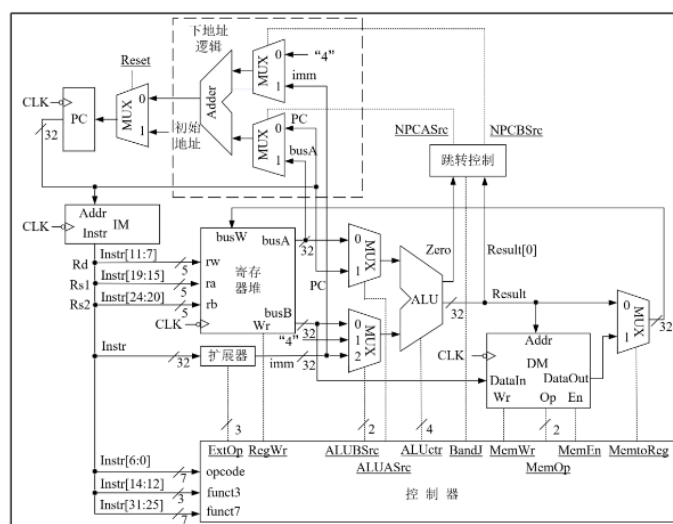


图 19: 数据通路原理图

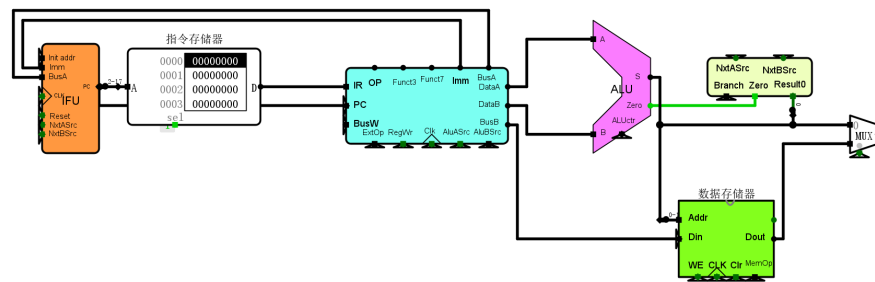
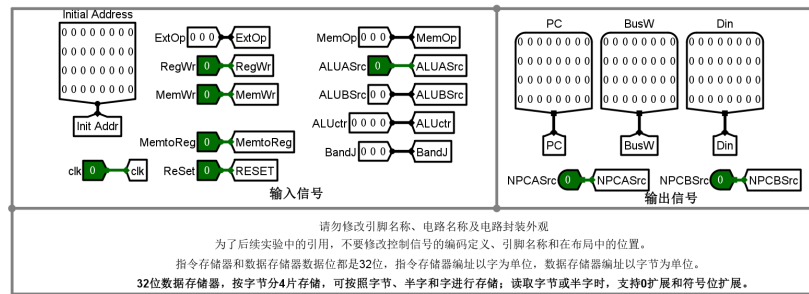


图 20: 主电路图

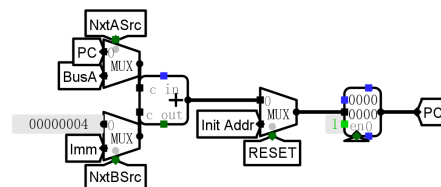
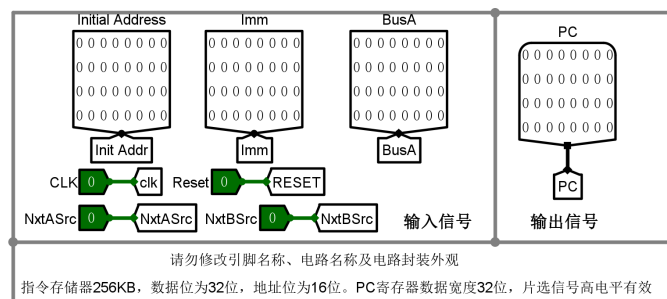


图 21: 取指令部件电路图

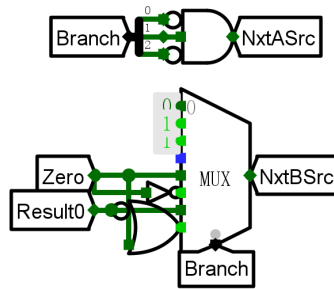
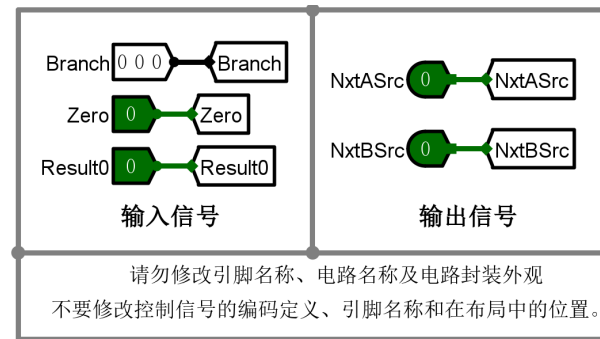
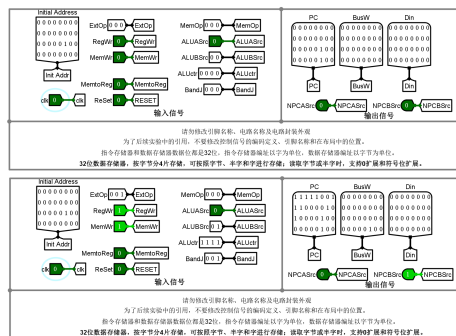
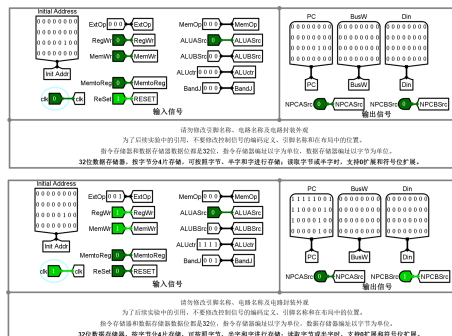


图 22: 分支跳转部件电路图

5.3 仿真测试



5.4 错误分析

做这个实验时，一开始忘记加载 lab5.5.hex 文件，导致整体电路无法正常工作；没有把 IFU 子电路中的 NPCASrc 和 NPCBSrc 隧道改为提供的 NxtASrc 和 NxtBSrc，导致测试无法通过。

6 思考题

滚动显示器

由于我们是按行输入，并且给到的 ASCII 码点阵显示表也是按照每个字节对应一行显示书写的，因此滚动的方向只能是从下往上滚动/从上往下滚动。

我把 read 子电路的 ROM 内容改成了如下所示：

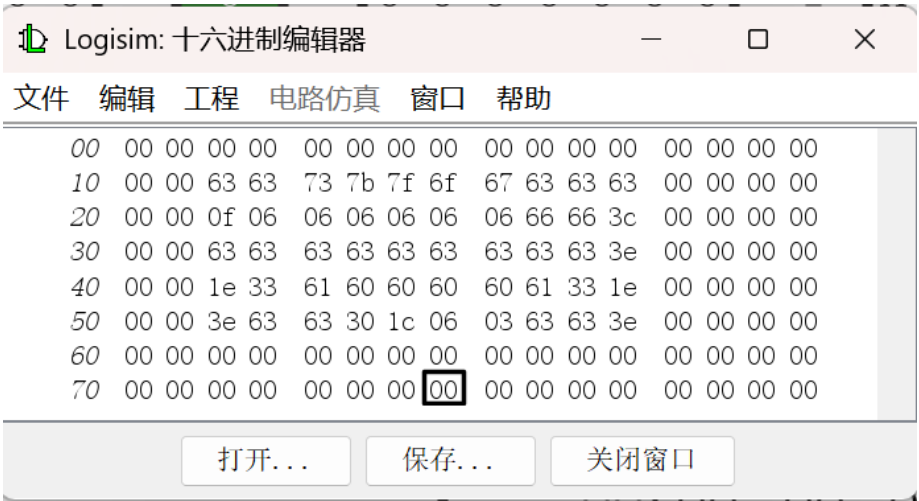


图 24: ROM 内容修改

其中第一行为空，第二至第六行为 NJUCS，后面均留空，这样只需要七位地址码，刚好和 ASCII 输入码位数相同，因此第一个 read 的输入直接就是 ASCII 码。地址每加一，就会输出下一行的点阵数据，这样就可以实现滚动显示了。

修改后的电路图如下：

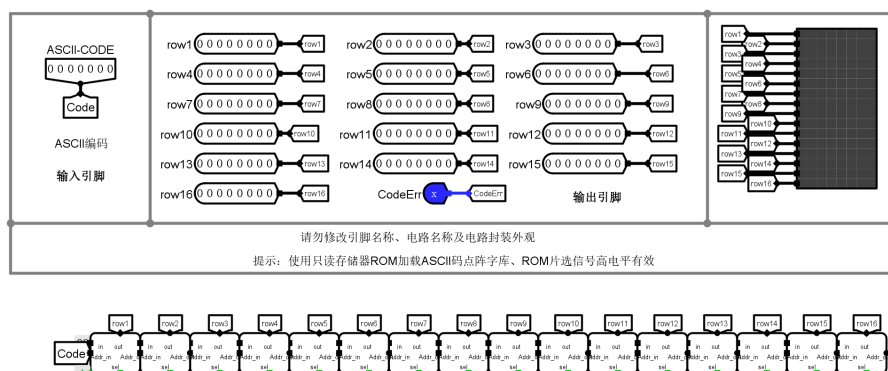


图 25: 修改后的 ASCII 显示电路图

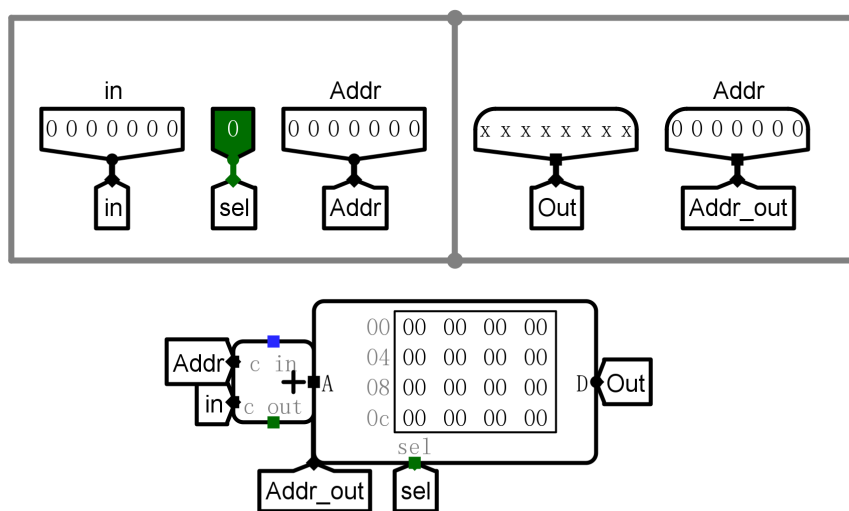


图 26: 修改后的 read 电路图

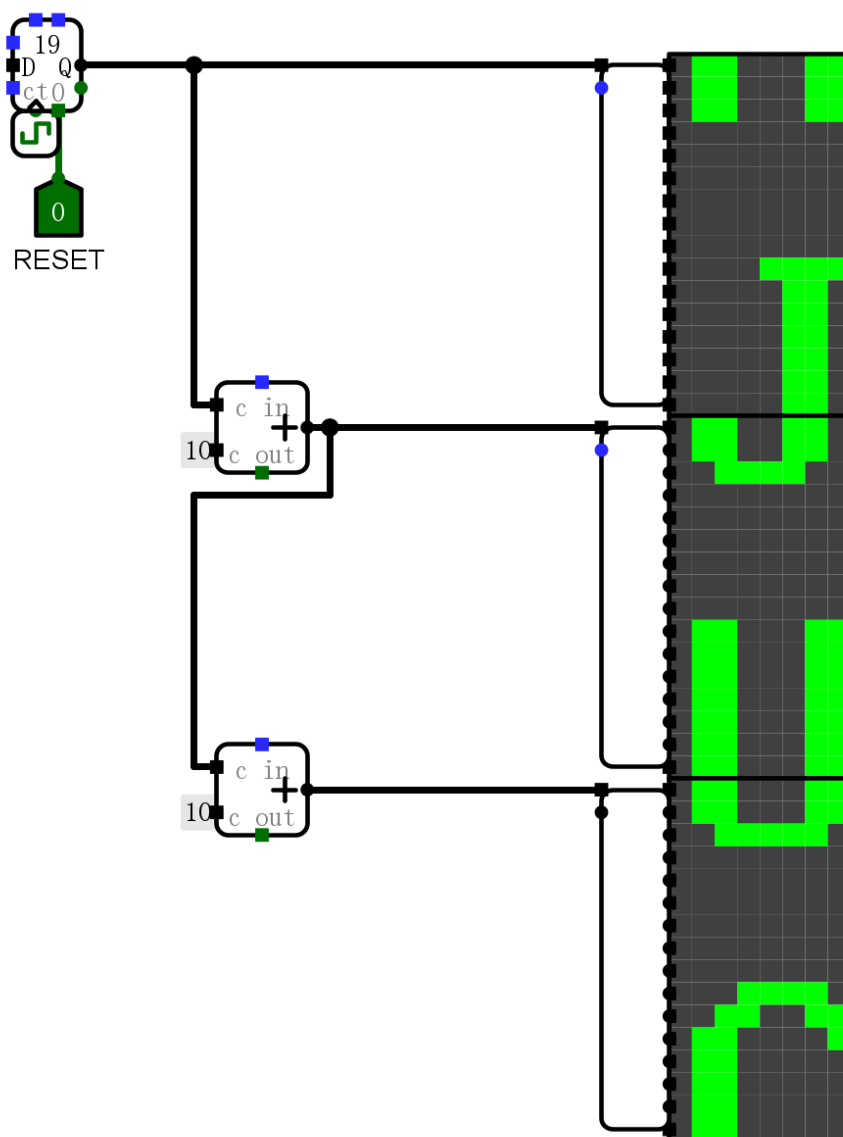


图 27: 修改后的主电路图

时钟信号触发边沿不一致对程序对执行结果的影响

可能导致存储器存入旧值；可能导致指令计数器 PC 和指令寄存器 IR 存入旧值或不正确的值，导致跳转错误；可能导致 ALU 操作数 A 和 B 存入旧值或不正确的值，导致计算错误；

可能导致寄存器堆写入错误的值，导致后续指令执行错误。

如何实现在当前程序结束后 CPU 不再继续往下执行

设计一条停止指令，当 CPU 执行到这条指令时，停止时钟信号的输入。